

A8 Multi-Agent Systems and Agentic AI

Funmi Looi-Somoye

Team members: Barbara Koch, Rowan d'Auria

24 June 2026

Contents

1	GRPO Finetuning	2
1.1	Reproducing the baseline	2
1.2	Understanding the baseline	2
1.3	Improving on the baseline	3
1.4	GRPO Theory	5
1.4.1	Question 1	5
1.4.2	Question 2	6
1.4.3	Question 3	7
2	Adaptive Planning	9
2.1	Workflow diagram	9
2.2	Problems	9
2.3	Proposed improvements	10
A	Appendix	12
A.1	Uncertainty for Table 1	12
A.2	Normalised reward component decomposition	12
A.3	Improving the baseline experiments	13
A.4	Part I.3 resources	15

1 GRPO Finetuning

1.1 Reproducing the baseline

Run details. On the team’s v6e-1 TPU, `scripts/train.py` at [commit 7e696c4](#) completed 3364 GRPO steps in 4 h 41 m 38 s and produced the final LoRA checkpoint. We used the default configuration (apart from the dependency patch below) and seed 42 for training shuffling and rollout sampling. The code is available at [tpu-2026](#) and the logs at [W&B jgs4c6k1](#).

Evaluation. Both the base model and the finetuned checkpoint were evaluated with `scripts/evaluate.py` using greedy decoding on the baseline’s held-out [GSM8K](#) test split, using 64 questions loaded through TFDS with shuffle seed 42. Three metrics are reported throughout. Exact numeric match is the fraction whose extracted answer equals the reference answer under the `match_numbers` extractor, which requires an `<answer>` tag but not the complete strict template. Within 10% numeric match is the fraction whose extracted number is within 10% of the reference answer. Strict template format is the fraction whose complete response template parses. The base [gemma-3-1b-it](#) solved 33/64 questions with 51.56% exact numeric match, 53.12% within 10% numeric match and 6.25% strict template format. The final LoRA checkpoint at step 3364 solved only 2/64 with 3.12% exact numeric match, 6.25% within 10% numeric match and 12.50% strict template format. On the full 1319-question split the base model scores 47.38% (Table 1).

The final checkpoint scored approximately 16 times lower than the base model. Evaluation reward peaked early (step 450, Fig. 1) then declined, and the later retained checkpoints track that decline. The final policy is effectively collapsed. The exact depth of this collapse is run specific, reflecting seed 42 and the full training budget rather than a fixed endpoint. A different seed or a shorter budget can retain more of the early competence. Checkpoints were saved every 500 steps with only the last four retained, so none from the early high reward window survived, motivating the stability work in I.3.

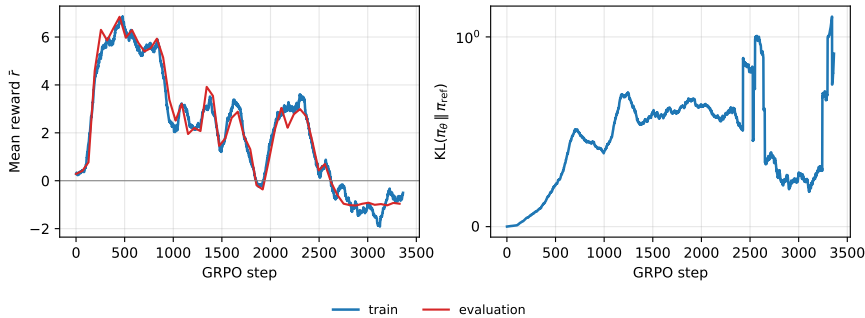


Figure 1: Baseline training curves (W&B run [jgs4c6k1](#)). The left panel shows mean rollout reward $\bar{r}$, with the 101 step centred moving average training trace in blue and raw held out evaluation in red. The right panel shows training $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$.

Baseline patches. The only patch pinned `protobuf==6.31.1` to fix a `tensorflow-datasets` build error. It is environment specific and alters no model, split, reward or hyperparameter.

1.2 Understanding the baseline

In GRPO, for each prompt q (a GSM8K question) a group of K responses $a_i \sim \pi_{\text{old}}(\cdot \mid q)$ is sampled and scored by a reward r_i . Each reward is centred on a group baseline $\bar{r}$ to form the normalised advantage $\hat{A}_i = (r_i - \bar{r})/(\sigma_r + \epsilon_{\text{std}})$. This advantage then drives a PPO-style clipped surrogate. Here ϵ_{std} is a numerical stabiliser, distinct from the clip range ϵ_{clip} in (d).

(a) Policy roles: π_θ is the trainable actor passed to Tunix. It is the [gemma-3-1b-it](#) base wrapped with LoRA adapters (`get_lora_model, actor=lora`), and only the LoRA adapter weights are trained. π_{ref} is the frozen base (`reference=base`), the anchor in the penalty $\beta \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$. π_{old} is the non-trainable policy that generated the current rollout batch. Since $\mu = \text{NUM_ITERATIONS} = 1$ there is a single

optimisation pass per rollout batch, so $\pi_{\text{old}} = \pi_{\theta}$ holds exactly at that one update and the per-token ratio is 1.

(b) Group size and advantage: The group size is $K = \text{NUM_GENERATIONS} = 2$. Tunix uses its default "grpo" estimator (`compute_advantages`), which subtracts the group mean $\bar{r}$ and divides by the group standard deviation (plus a 10^{-6} stabiliser). This is the standard advantage, not leave-one-out. Tunix's own rloo estimator returns the unnormalised $r_i - \bar{r}_{-i}$, related by $r_i - \bar{r} = \frac{K-1}{K}(r_i - \bar{r}_{-i})$, and the additional division by the sample standard deviation is what discards the reward gap magnitude. At $K = 2$ this is most pronounced. With two samples the standard deviation is proportional to $|r_1 - r_2|$, so the normalised advantage reduces to $\hat{A}_i \propto \text{sign}(r_i - r_j)$ with a fixed magnitude independent of the gap, and each group contributes only a binary contrastive sign.

(c) Reward terms: `match_format_exactly` and `match_format_approximately` are shaping rewards on the `<reasoning>...</reasoning><answer>...</answer>` template. The true correctness reward is `check_answer` (+3.0 exact, with graded partial credit), with `check_numbers` (+1.5) the number extraction fallback. They are not interchangeable. `check_answer` extracts via `match_format` and returns 0 whenever the template fails to parse, coupling correctness to formatting. The headline evaluation metric instead uses `match_numbers` with float equality. This requires the `<answer>` tag but does not require the complete strict template. With correctness alone both completions typically score 0 early, so $\sigma_r \approx 0$ and the advantage carries almost no signal. The shaping rewards increase the probability of nonzero within group variance early in training, while rare correct groups collapse to the fixed binary contrastive advantages described in (b).

(d) Clipped surrogate: The clipped surrogate is the registered grpo policy loss, with clip range $\epsilon_{\text{clip}} = \text{EPSILON} = 0.2$. It implements $\min(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) \hat{A}_i)$. There are three deviations from the lecture form. (1) The lecture uses a sequence level ratio $\rho_i = \pi_{\theta}(a_i | q) / \pi_{\text{old}}(a_i | q)$. Tunix instead clips per-token ratios, with the sequence level advantage broadcast over tokens. The aggregation is `loss_agg_mode = "sequence-mean-token-mean"`, so each completion is weighted equally and the length bias of a flat token mean is avoided. (2) Most importantly, with $\mu = 1$ the single pass leaves no gap between π_{old} and π_{θ} , so $\rho_i = 1$ identically and lies inside $[1 - \epsilon, 1 + \epsilon]$. The clip therefore never binds and $\epsilon = 0.2$ has essentially no effect. (3) The β -KL to the reference is a per-token penalty inside the loss (k3 estimator), `BETA = 0.08 (config.py:48)`. Although the forward ratio equals one, its denominator is stop gradient, so gradients still flow through the numerator. Clipping does not constrain every ratio to remain inside the interval. For a positive advantage it removes further incentive above $1 + \epsilon$. For a negative advantage it removes further incentive below $1 - \epsilon$.

1.3 Improving on the baseline

Modification and hypothesis. Our principled change raises the group size from $K = 2$ to $K = 8$. At $K = 2$ the normalised advantage retains only the sign of the reward gap, and tied rewards give no update. This sign-only advantage discards the reward-gap magnitude. It also leaves only $K_{\text{eff}} = K - 1 = 1$ effective sample per group. We therefore expected $K = 8$ to give a graded advantage with $K_{\text{eff}} = 7$ and lower variance. Fewer tied groups may also help, but shaping keeps $\sigma_r > 0$ at $K = 2$, so this is secondary. This comparison uses more rollout compute per step and does not by itself establish that lower gradient variance caused the result.

Our next modification targets the reward inside the stable $K = 8$ regime. The component scores show that strict template format was already near saturation while exact numeric match remained lower. This motivated two changes. We shrank the format terms to a gated entry fee with budget 1.0. We then replaced the discontinuous `check_answer` ladder with a smooth curve $r_{\text{corr}} = \text{CORRECT_MAX} e^{-\text{CORRECT_K} \epsilon}$ in the relative error ϵ with budget 5.5, so approximately correct answers receive graded reward.

Setup. All three runs used $\text{lr } 3 \times 10^{-6}$, $\beta = 0.08$, seed 42 and 5864 steps. We extended all three runs from the default (3364) to 5864 steps so that the reward experiment, which branches from the shared trunk at step 3364, had a comparable post-branch training window. The runs were the $K = 2$ baseline (`keh7f5es`, `baseline-k=2`), $K = 8$ with the baseline reward (`m3xp6k97`, `n-generations-8-rerun`),

and reward-reweighted $K = 8$ (smuzmoal, reward-reweight). The $K = 8$ runs share a trunk to step 3364, isolating the subsequent reward change. We evaluated each checkpoint greedily on a seed-42, 64-question subset and final checkpoints on all 1319 test questions with bootstrap CIs (Table 1). The full split was held out from training, but it was not completely independent of model selection because the subset came from the same split. Comparisons are per step rather than compute matched, so some $K = 8$ gain may reflect its greater rollout throughput. Proportionally more $K = 2$ steps would better isolate the effect of K .

Model	Step	Seed	Correct	Accuracy (exact numeric)	Δ vs. base
Base gemma-3-1b-it	—	—	625/1319	47.38 [44.81, 50.11]	—
Baseline GRPO ($K=2$)	5864	42	1/1319	0.08 [0.00, 0.23]	−47.31 [−50.04, −44.66]***
$K=8$	5864	42	739/1319	56.03 [53.45, 58.76]	+8.64 [+5.84, +11.45]***
$K=8$ + reweighted reward	5864	42	772/1319	58.53 [55.88, 61.18]	+11.14 [+8.42, +13.80]***

Table 1: Full split evaluation with greedy decoding on all $n = 1319$ GSM8K test questions, seed 42. Accuracy is the exact numeric match percentage, and brackets give 95% bootstrap CIs (paired for Δ vs. base). *** $p < 0.001$ (McNemar test, Holm corrected). All three gains are significant, while the reward reweighting pairwise comparison is not (+2.50 pp, $p = 0.064$). Per-model within-10% and strict-format rates are in Appendix Table 2.

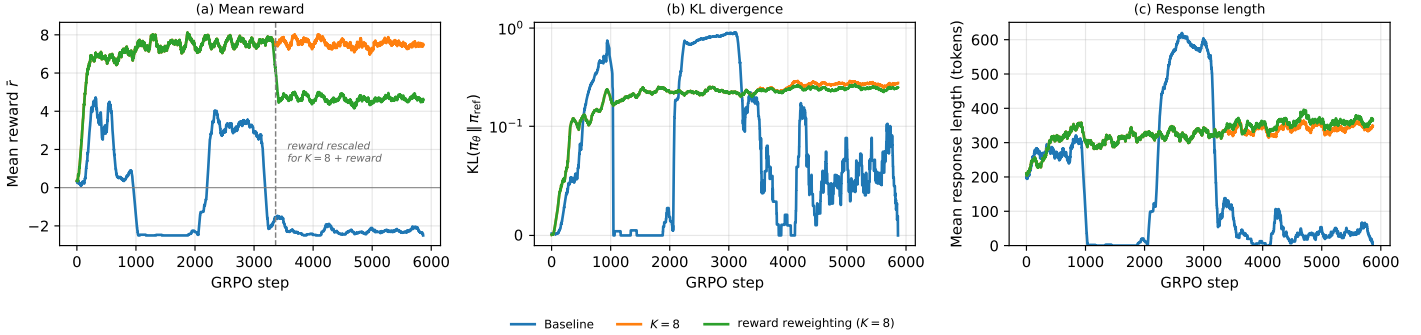


Figure 2: Training curves for the $K = 2$, $K = 8$, and reward reweighted $K = 8$ runs. The panels show mean rollout reward, $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ and centred moving average response length. The $K = 8$ runs share a trunk to step 3364. Post branch rewards are not directly comparable because the reward scale changes.

Discussion. Against the $K = 2$ GRPO baseline, $K = 8$ avoids collapse and finishes 55.95 points higher with paired 95% CI [53.37, 58.68]. Against the base model, $K = 8$ improves exact numeric match by 8.64 points with paired 95% CI [5.84, 11.45]. The training curves show that $K = 2$ collapses while $K = 8$ maintains a stable KL and response length (Fig. 2). The likely mechanism is the advantage itself. At $K = 8$ it is graded, not sign-only. The effective sample size rises from 1 to 7 which lowers the gradient variance behind the $K = 2$ collapse. The stable KL and length agree. Fewer tied groups may also contribute. However, $\sigma_r > 0$ at $K = 2$, so ties are secondary and were not measured. The reward update begins after step 3364 on the shared $K = 8$ trunk. Normalised component trajectories provide supplementary evidence that format scores were near saturation while correctness scores remained lower (Appendix Fig. 4). Logged magnitudes largely reflect the altered reward scale, while normalised trajectories show little movement. We predicted that replacing the discontinuous `check_answer` ladder with a smooth graded reward would raise exact numeric match by crediting approximately correct answers. This was not the case, however. Exact numeric match rises by only 2.50 points with paired 95% CI [−0.08, 5.00] and Holm adjusted $p = 0.064$, which is suggestive but not statistically significant. Each run used one seed, so the intervals describe test set sampling uncertainty rather than training variability. With more compute time a second seed run would be performed to assess reproducibility. Further experiments are reported in the Appendix, including a harder GSM8K variant that trains only on the hard and medium difficulty buckets rather than the full distribution.

1.4 GRPO Theory

1.4.1 Question 1

Throughout this question we write σ for σ_r .

(i) Since h_i is deterministic and only $r_1, \dots, r_K$ are random,

$$\mathbb{E}[X_i] = h_i \mathbb{E}[\hat{A}_i] = \frac{h_i}{\sigma} \mathbb{E}[r_i - \bar{r}] = \frac{h_i}{\sigma} (\mu - \mu) = 0, \quad (1)$$

and so $\mathbb{E}[\hat{g}] = \frac{1}{K} \sum_{i=1}^K \mathbb{E}[X_i] = 0$.

The expectation here only averages over the reward noise, with the responses $a_1, \dots, a_K$ held fixed. So $\mathbb{E}[\hat{g}] = \mathbf{0}$ indicates only that the group baseline is unbiased; it does not imply the policy has stopped improving. The true policy gradient $\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a | q) A^{\pi_\theta}(q, a)]$ averages over the actions the policy actually samples, and is nonzero as long as different actions have different advantages.

(ii) Every X_i and X_j share the term $\bar{r} = \frac{1}{K} \sum_{k=1}^K r_k$. Since both means are zero,

$$\text{Cov}(X_i, X_j) = \mathbb{E}[X_i X_j^\top] = h_i h_j^\top \frac{\mathbb{E}[(r_i - \bar{r})(r_j - \bar{r})]}{\sigma^2}, \quad i \neq j. \quad (2)$$

Using independence of $r_1, \dots, r_K$ and standard covariance properties,

$$\begin{aligned} \mathbb{E}[(r_i - \bar{r})(r_j - \bar{r})] &= \text{Cov}(r_i, r_j) - \text{Cov}(r_i, \bar{r}) - \text{Cov}(\bar{r}, r_j) + \text{Var}(\bar{r}) \\ &= 0 - \frac{\sigma^2}{K} - \frac{\sigma^2}{K} + \frac{\sigma^2}{K} = -\frac{\sigma^2}{K}. \end{aligned} \quad (3)$$

Hence,

$$\text{Cov}(X_i, X_j) = -\frac{1}{K} h_i h_j^\top, \quad i \neq j. \quad (4)$$

The sign is negative because a high draw of r_i raises $\bar{r}$, which lowers $\hat{A}_j = (r_j - \bar{r})/\sigma$ for every other j . A high reward for one response lowers the relative advantage of all the others. This is why the K samples in a group are not statistically independent, as the negative off diagonal terms partly cancel the diagonal of $\text{Var}(\hat{g})$.

(iii) Noting that $\text{Var}(r_i - \bar{r}) = \sigma^2 - 2\sigma^2/K + \sigma^2/K = \sigma^2(1 - 1/K)$, the diagonal terms are

$$\text{Var}(X_i) = h_i h_i^\top \frac{K-1}{K}. \quad (5)$$

Let $\bar{h} = \frac{1}{K} \sum_i h_i$ and $H = K\bar{h}$. Collecting diagonal and off-diagonal contributions and using $\sum_{i \neq j} h_i h_j^\top = HH^\top - \sum_i h_i h_i^\top$,

$$\begin{aligned} \text{Var}(\hat{g}) &= \frac{1}{K^2} \left[\frac{K-1}{K} \sum_i h_i h_i^\top - \frac{1}{K} \left(HH^\top - \sum_i h_i h_i^\top \right) \right] \\ &= \frac{1}{K^2} \left[\sum_i h_i h_i^\top - \frac{1}{K} HH^\top \right] = \frac{\Sigma_h}{K}, \end{aligned} \quad (6)$$

where $\Sigma_h = \frac{1}{K} \sum_i h_i h_i^\top - \bar{h} \bar{h}^\top$ is the empirical covariance matrix of the score vectors $\{h_i\}$.

If the K samples were i.i.d. copies of X_1 we would instead get $\text{Var}(\hat{g})_{\text{naive}} = \text{Var}(X_1)/K = \frac{K-1}{K^2} h_1 h_1^\top$. This depends only on h_1 , while the true variance in (6) depends on the centred spread of all score vectors. The matrices are not generally proportional, so a scalar effective sample size depends on the

chosen projection. For a direction $u \in \mathbb{R}^d$, let $s_i = u^\top h_i$ and $\bar{s} = K^{-1} \sum_i s_i$. Then

$$\text{Var}(u^\top \hat{g}) = \frac{1}{K^2} \sum_{i=1}^K (s_i - \bar{s})^2. \quad (7)$$

Using $\text{Var}(u^\top X_1)$ as the one sample reference gives

$$K_{\text{eff},1}(u) = \frac{\text{Var}(u^\top X_1)}{\text{Var}(u^\top \hat{g})} = \frac{K(K-1)s_1^2}{\sum_i (s_i - \bar{s})^2}. \quad (8)$$

Its value depends on which response is used as the scale. A symmetric convention uses $\bar{V}_X(u) = K^{-1} \sum_i \text{Var}(u^\top X_i)$ and defines

$$K_{\text{eff}}(u) = \frac{\bar{V}_X(u)}{\text{Var}(u^\top \hat{g})} = \frac{(K-1) \sum_i s_i^2}{\sum_i (s_i - \bar{s})^2}. \quad (9)$$

For centred projected scores with $\bar{s} = 0$, this convention gives $K_{\text{eff}}(u) = K - 1$. The relative cost of estimating the group baseline is therefore largest for small K , while $K_{\text{eff}}(u)/K \rightarrow 1$ for large K .

If all score vectors are aligned with $h_i = c_i \mathbf{v}$, only variation in the coefficients c_i survives the centring in (7). Less variation in c_i gives lower variance, but it may also give a less useful gradient signal. If every c_i is equal, then $\sum_i \hat{A}_i = 0$ implies $\hat{g} = 0$ exactly. This is a zero variance degeneracy rather than evidence of high estimator efficiency. It is distinct from an all equal reward group, which also gives no update because every normalised advantage is zero.

1.4.2 Question 2

This question looks at the idealised KL-regularised update ($\dagger$), why it guarantees improvement, and the clipped surrogate GRPO uses instead.

(i) Form the Lagrangian with multiplier λ enforcing $\sum_a \pi(a) = 1$,

$$\mathcal{L}(\pi, \lambda) = \sum_a \pi(a) A_t(a) - \frac{1}{\eta} \sum_a \pi(a) \log \frac{\pi(a)}{\pi_t(a)} + \lambda \left(\sum_a \pi(a) - 1 \right). \quad (10)$$

Setting $\partial \mathcal{L} / \partial \pi(a) = 0$ and solving gives the unique optimiser on the support of π_t

$$\pi^*(a \mid q) = \frac{\pi_t(a) \exp(\eta A_t(a))}{Z_t(\eta)}, \quad Z_t(\eta) = \sum_{a'} \pi_t(a') \exp(\eta A_t(a')). \quad (11)$$

This reweights π_t by $\exp(\eta A_t)$, trading a higher advantage against staying close to π_t in KL. The parameter η acts like a step size and sets how far the update can move from π_t . As $\eta \rightarrow 0$ we get $\pi^* \rightarrow \pi_t$ and the policy is unchanged, while as $\eta \rightarrow \infty$ all the mass concentrates on $\arg \max_a A_t(a)$, a greedy collapse that eliminates exploration. An action with $\pi_t(a) = 0$ cannot acquire positive probability because that would give infinite forward KL.

(ii) Define $J(\pi) = \mathbb{E}_{a \sim \pi} [A^{\pi_t}(a)]$. With exact advantages $A_t = A^{\pi_t}$, the standard advantage identity gives

$$J(\pi_t) = \mathbb{E}_{a \sim \pi_t} [A^{\pi_t}(a)] = \mathbb{E}_{a \sim \pi_t} [Q^{\pi_t}(a) - V^{\pi_t}] = V^{\pi_t} - V^{\pi_t} = 0. \quad (12)$$

Let $F(\pi) = \mathbb{E}_\pi [A_t] - \frac{1}{\eta} \text{KL}(\pi \parallel \pi_t)$ denote the ($\dagger$) objective. Evaluating at the current iterate and using (12),

$$F(\pi_t) = J(\pi_t) - \frac{1}{\eta} \text{KL}(\pi_t \parallel \pi_t) = 0. \quad (13)$$

Since π_{t+1} maximises F , it follows that $F(\pi_{t+1}) \geq F(\pi_t) = 0$, which gives

$$J(\pi_{t+1}) \geq \frac{1}{\eta} \text{KL}(\pi_{t+1} \| \pi_t) \geq 0 = J(\pi_t), \quad (14)$$

establishing monotone improvement.

This guarantee only holds for the idealised ($\dagger$) step. The practical update optimises a finite sample clipped surrogate built from noisy advantages rather than exactly maximising the true KL regularised objective. Therefore the comparison $F(\pi_{t+1}) \geq F(\pi_t)$ is not guaranteed. The identity $J(\pi_t) = 0$ still holds because J is defined using the true advantage.

(iii) For a sample with $\hat{A}_t > 0$, clipping removes further incentive once $\rho_t(\theta) > 1 + \varepsilon$. For $\hat{A}_t < 0$, it removes further incentive once $\rho_t(\theta) < 1 - \varepsilon$. It does not impose a hard constraint on either ratio. Parameter sharing and later optimisation steps can still move sampled ratios beyond these thresholds, while unsampled actions are not directly controlled. The KL constraint instead bounds a single probability weighted divergence over the whole action distribution. PPO clipping therefore gives a local one sided modification of the sampled objective, while the KL constraint defines a joint trust region for the policy.

1.4.3 Question 3

This question estimates the GRPO gradient from samples drawn under a mixture proposal, using importance sampling (IS) to keep the estimator targeted at π_{old} .

(i) The target is the policy gradient under π_{old} ,

$$g = \mathbb{E}_{a \sim \pi_{\text{old}}(\cdot | q)} \left[\nabla_{\theta} \log \pi_{\theta}(a | q) \cdot \hat{A}(a) \right]. \quad (15)$$

Rewriting via importance sampling, $\mathbb{E}_{\pi_{\text{old}}}[f] = \mathbb{E}_{\pi_{\text{mix}}}[(\pi_{\text{old}}/\pi_{\text{mix}}) f]$, the corrected estimator is

$$\hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^K w_i \nabla_{\theta} \log \pi_{\theta}(a_i | q) \cdot \hat{A}_i, \quad (16)$$

where the importance weight for sample i is

$$w_i = \frac{\pi_{\text{old}}(a_i | q)}{\pi_{\text{mix}}(a_i | q)} = \frac{\pi_{\text{old}}(a_i | q)}{(1 - \alpha) \pi_{\text{old}}(a_i | q) + \alpha \pi_{\text{unif}}(a_i | q)}. \quad (17)$$

The identity retargets the expectation onto π_{old} while sampling from π_{mix} . The proposal must cover the target support. This is guaranteed by the specified range $\alpha < 1$ because the mixture retains positive mass from π_{old} .

(ii) As $\alpha \rightarrow 0$, $\pi_{\text{mix}} \rightarrow \pi_{\text{old}}$, so for every realised a_i , $w_i \rightarrow \pi_{\text{old}}(a_i | q) / \pi_{\text{old}}(a_i | q) = 1$. Substituting into (16),

$$\hat{g}_{\text{mix}} \xrightarrow{\alpha \rightarrow 0} \frac{1}{K} \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i | q) \cdot \hat{A}_i = \hat{g}_{\text{GRPO}}. \quad (18)$$

This confirms $\hat{g}_{\text{mix}}$ is a consistent generalisation of standard GRPO. We recover GRPO exactly as the proposal returns to π_{old} . It is also the equality case ($\alpha = 0$) for the variance and weight results below.

(iii) (a)

$$V^{\pi_{\text{mix}}}(q) = \mathbb{E}_{a \sim \pi_{\text{mix}}}[R(q, a)] = (1 - \alpha) V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q). \quad (19)$$

(b) Since $a_1, \dots, a_K \sim \pi_{\text{mix}}$, the sample mean $\bar{r} = \frac{1}{K} \sum_i r_i$ estimates $V^{\pi_{\text{mix}}}(q)$, not $V^{\pi_{\text{old}}}(q)$. The importance weights w_i in (17) correct the score-function weighting, but they leave the baseline

unchanged. So the advantage $\hat{A}_i = (r_i - \bar{r})/\sigma_r$ still centres rewards on $V^{\pi_{\text{mix}}}$ rather than $V^{\pi_{\text{old}}}$.

A practical importance sampled estimate of $V^{\pi_{\text{old}}}(q)$ from the mixture samples is the self-normalised baseline

$$\hat{b}_{\text{old}} = \frac{\sum_i w_i r_i}{\sum_i w_i}, \quad (20)$$

which is consistent but not exactly unbiased at finite K , since it is a ratio of two random sums and sample i enters both the baseline and its own score term, leaving an $O(1/K)$ bias in the gradient. An exactly unbiased baseline instead uses the ordinary importance sampling leave-one-out mean

$$\hat{b}_{\text{old},-i} = \frac{1}{K-1} \sum_{j \neq i} w_j r_j. \quad (21)$$

Because $\mathbb{E}_{\pi_{\text{mix}}}[w_j] = 1$ and $\mathbb{E}_{\pi_{\text{mix}}}[w_j r_j] = V^{\pi_{\text{old}}}(q)$, this mean has expectation $V^{\pi_{\text{old}}}(q)$ exactly, and dropping sample i keeps it independent of a_i . The reweighted advantage that keeps the gradient unbiased for the policy gradient under π_{old} is then

$$\tilde{A}_i = w_i \frac{r_i - \hat{b}_{\text{old},-i}}{\sigma_r}, \quad \hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^K \nabla_{\theta} \log \pi_{\theta}(a_i | q) \tilde{A}_i, \quad (22)$$

where the scale σ_r is the reward standard deviation under π_{old} , taken as fixed or estimated independently of sample i , since standardising by a sample dependent scale would reintroduce a finite sample bias. Here $\tilde{A}_i$ absorbs the importance weight. It replaces the whole product $w_i \hat{A}_i$ in (16), not just $\hat{A}_i$, so the weight is not applied twice. Reweighting the score alone is not enough. We must also recentre on the leave-one-out baseline $\hat{b}_{\text{old},-i}$. By (19) the leftover gap is $V^{\pi_{\text{mix}}} - V^{\pi_{\text{old}}} = \alpha(V^{\pi_{\text{unif}}} - V^{\pi_{\text{old}}})$, which grows linearly in α . Without the corrected baseline, $\hat{g}_{\text{mix}}$ stays centred on $V^{\pi_{\text{mix}}}$ and is biased for the π_{old} gradient.

(iv) The raw reward variance under the mixture is

$$\begin{aligned} \text{Var}_{\pi_{\text{mix}}}(R) &= (1 - \alpha)\text{Var}_{\pi_{\text{old}}}(R) + \alpha\text{Var}_{\pi_{\text{unif}}}(R) \\ &\quad + \alpha(1 - \alpha)(V^{\pi_{\text{old}}} - V^{\pi_{\text{unif}}})^2. \end{aligned} \quad (23)$$

Subtracting $\text{Var}_{\pi_{\text{old}}}(R)$ shows that mixing increases raw reward variance when

$$\text{Var}_{\pi_{\text{unif}}}(R) - \text{Var}_{\pi_{\text{old}}}(R) + (1 - \alpha)(V^{\pi_{\text{old}}} - V^{\pi_{\text{unif}}})^2 > 0. \quad (24)$$

It decreases raw reward variance when the inequality is reversed. The first case occurs when the uniform component has greater reward variance or a sufficiently different mean. The second case occurs when the old policy has greater reward variance and the component means are similar. The gradient estimator also contains the importance weights. Unequal weights reduce $N_{\text{eff}} = (\sum_i w_i)^2 / \sum_i w_i^2$, so weight dispersion can overturn a reduction in raw reward variance. Lower gradient variance therefore requires both a favourable reward distribution and controlled importance weights.

(v) (a) When $0 < c < 1$, the weight $w_i = c^T$ decays geometrically to zero as $T \rightarrow \infty$. A value $c < 1$ is typical once $\alpha > 0$. Under π_{mix} the per-token ratio $\pi_{\text{old}}/\pi_{\text{mix}}$ has arithmetic mean exactly 1. The product over T tokens is instead determined by its geometric mean, $\exp(\mathbb{E}_{\pi_{\text{mix}}}[\log(\pi_{\text{old}}/\pi_{\text{mix}})]) = \exp(-\text{KL}(\pi_{\text{mix}} \parallel \pi_{\text{old}}))$. This sits below 1 whenever π_{mix} differs from π_{old} , hence for any $\alpha > 0$.

(b) For long responses $w_i = c^T \rightarrow 0$, so their raw gradient contributions vanish. In practice the ratios differ across responses, which produces highly unequal weights and allows a small number of rollouts to dominate. One concrete fix is self normalisation

$$\tilde{w}_i = \frac{w_i}{\sum_{j=1}^K w_j}, \quad \hat{g}_{\text{mix}} = \sum_{i=1}^K \tilde{w}_i \nabla_{\theta} \log \pi_{\theta}(a_i | q) \hat{A}_i, \quad (25)$$

which prevents a common shrinkage factor from making the estimate vanish. It introduces finite sample bias and does not repair severe weight inequality. Reducing α is another option. It limits weight dispersion but also reduces exploration.

2 Adaptive Planning

The exact implementation studied is the frozen `adaptive-planning` branch of `cmbagent_lg` at commit [d7d0592](#). The critique draws on the `adaptive_planning` module and the `planning`, `deep_research` and `self_debug` modules it composes.

2.1 Workflow diagram

The adaptive-planning workflow is shown in Figure 3.

Walkthrough

Trigger. The initial `Plan` comes from the separate `planning` proposal and critique loop. The adaptive graph then executes it one step at a time. `execute_step`, which reuses `deep_research.run_step`, appends a summary and outcome and increments `step_index`. It then passes unconditionally to `adaptive_review`, even when execution failed. The reviewer reads only the latest summary, latest outcome and remaining tail. `route_after_review` rewrites the plan only when `needs_adaptation` is `True` and neither the failure nor exhaustion check has already selected `END`.

What changes and what remains fixed. Only the unexecuted tail changes. `replan_tail` regenerates it in one planner pass, so step count, order, instructions, timeouts and agents may change subject to the valid agent schema. The implementation computes `n_done = step_index - 1` and holds the first `n_done` tasks fixed. `merge_plan` writes both the merged plan and `adaptive_history`. The completed summaries, outcomes, step index and global task remain fixed.

Termination. The current graph reaches the same `END` node after failed execution, successful plan completion or an empty replanned tail. These outcomes are not recorded as distinct terminal states. Between revisions, `execute_step` runs and increments `step_index`. Revision count and tail length remain uncapped, so the only hard backstop is `LangGraph`'s default `recursion_limit`, which raises an exception. Termination is therefore conditional rather than guaranteed, and there is no explicit budget exhaustion outcome.

2.2 Problems

Three significant problems are argued in depth from the implementation, followed by a brief fourth problem.

Problem 1. Adaptation can only follow a successful step. `route_after_review` returns `END` whenever the last outcome has `fulfilled=False`, so the plan is rewritten only after a successful step. A failed step still incurs the review call, but its verdict is then discarded. The lectures define closed loop planning as revising the plan in response to observed outcomes. By the time control returns here, `self_debug` has exhausted local recovery, so the adaptive layer should be able to reroute, insert a data acquisition step or remove an impossible downstream step. Instead it halts where top level adaptation is most valuable.

Problem 2. The tail rewrite is unreviewed and not dependency validated. The initial plan uses separate proposal and critique, but `replan_tail` uses one generator pass and `merge_plan` concatenates prefix and tail without validation. No node checks the rewritten tail against the fixed prefix, latest outcome, workspace manifest or earlier revisions. `adaptive_history` is written but never read, so revisions can oscillate. Later tasks depend on earlier summaries and workspace artefacts, which means a rewritten step may expect a stale file, schema or result.

Problem 3. Plan drift and cost are unbounded. Nothing bounds the number of revisions or the length of `new_tail`. The setting `maximum_number_of_steps_in_plan` shapes only the initial prompt and does not constrain the adaptive graph. Every step adds a critic call and every adaptation adds a generator call without a cost, latency or capability budget. The loop records node duration but never uses it as a control signal. A long run can therefore end with `GraphRecursionError` rather than a controlled budget outcome. An unconstrained replan can also return an empty tail, causing `route_after_merge` to end the run without distinguishing this case from successful completion.

Problem 4. The adaptation signal is thin. `adaptive_review` sees only the latest summary, latest outcome and remaining tail. It does not see earlier summaries, the workspace manifest or the actual artefacts. The router acts on the `needs_adaptation` boolean without severity or confidence. On the unsupported agent path, `run_step` appends an outcome without appending a summary, so the reviewer can pair the current outcome with the previous summary. The decision node is also less informed than the replanning node, which receives all completed summaries.

2.3 Proposed improvements

The improvements below address the problems of II.2, each stating what it changes, why it addresses the problem, and what it trades off.

Problem 1. Route before review. Add a router immediately after `execute_step`. Successful execution proceeds to `adaptive_review`. Failed execution proceeds directly to a recovery replan bounded by `max_failure_replans`. Since execution increments `step_index`, the implementation value `n_done = step_index - 1` includes the failed task. Recovery must therefore retain only the fulfilled prefix and place the failed task back at the start of the mutable tail. The trade off is extra recovery cost and a risk of masking a genuinely infeasible path.

Problem 2. Split adaptation into specialised roles. Add optional logical `requires[]` and `produces[]` fields to `Step`. A Decision Agent should audit dependencies against the fulfilled prefix, latest outcome, workspace manifest and `adaptive_history`. An `update_plan` agent should rewrite only affected tail steps. A `tail_critic` should review the proposal for at most `max_tail_critic_rounds`. A deterministic `validate_merge` node should reject schema and control errors. The Decision Agent should emit broken dependencies, severity, confidence and an action. This grounds adaptation in whether dependencies remain valid. The trade off is additional nodes and model calls.

Problem 3. Bound and route the controller. Limit adaptations to three, failure replans to one and critic rounds to two. Limit total steps to $\min(2N, N + 5)$. Add a monetary budget and derive `recursion_limit` from these caps. The controller should return distinct outcomes for successful completion, failed termination, budget exhaustion and empty tail termination.

Use a small model (for example GPT-5 mini) for routine dependency extraction and tail generation. The Decision Agent should always emit a verdict. Use a larger reasoning critic (for example GPT-5.5) only when severity is high or decision confidence is below 0.7. Use no model for deterministic merge validation. The trade off is that caps can stop a useful late adaptation and model routing adds configuration.

Problem 4. Widen and structure the adaptation signal. The Problem 2 redesign already removes most of this sparsity. The Decision Agent now reads the fulfilled prefix, workspace manifest and `adaptive_history`, not just the latest summary and outcome. It also emits severity, confidence and an action in place of the bare `needs_adaptation` boolean. Two further changes would complete the fix. First, we extend that context to the full set of completed summaries the replanner already receives, so the decision node is no less informed than the replanner. Second, we repair the unsupported agent path so `run_step` appends a summary alongside every outcome, removing the summary and outcome desynchronisation. The trade off is larger reviewer prompts and a little additional implementation work on the unsupported agent path.

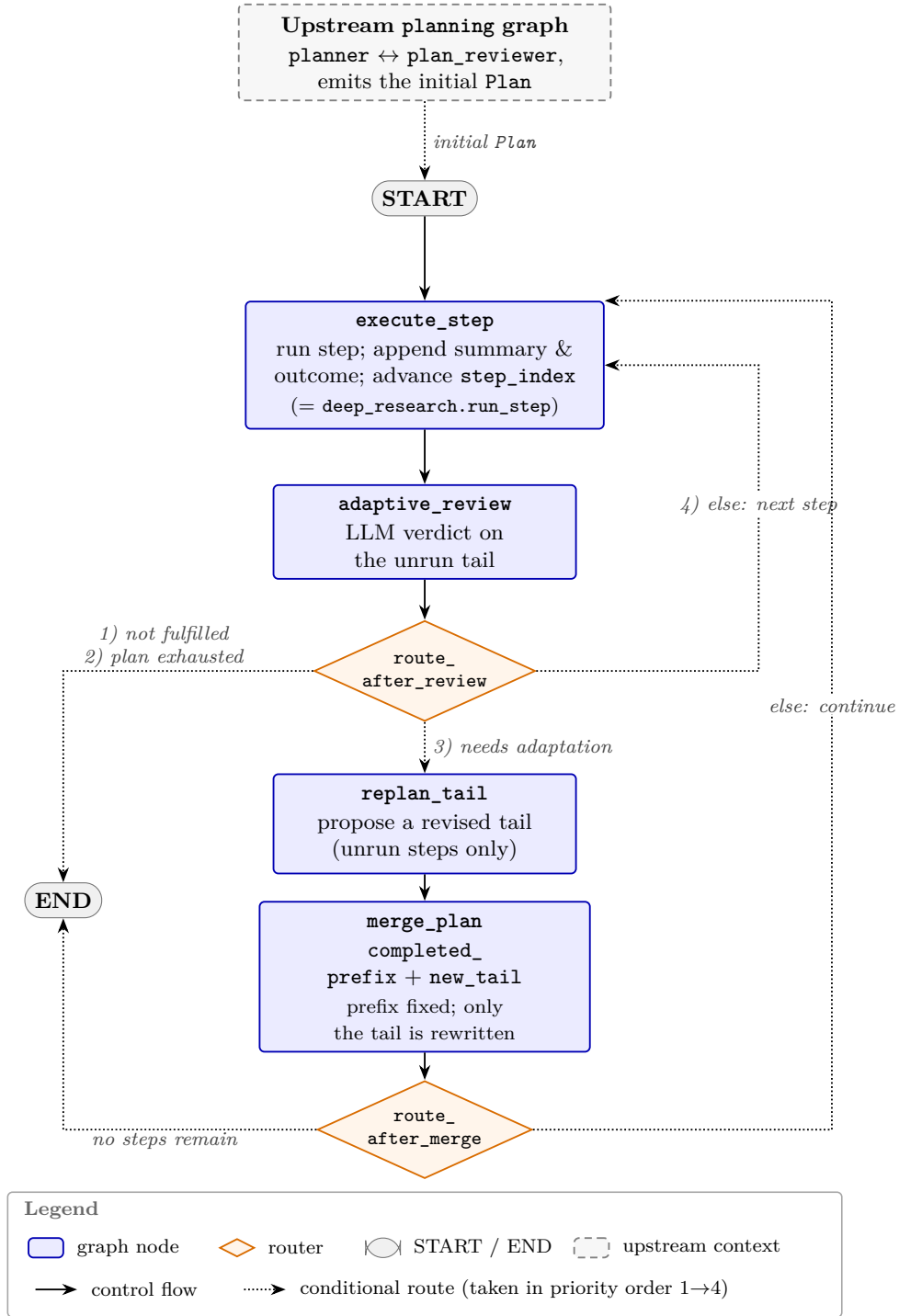


Figure 3: The complete workflow at commit d7d0592. The dashed box compresses the separate upstream **planning** graph, whose planner–reviewer loop produces the initial **Plan**; the remaining nodes are the implemented **adaptive_planning** graph. Diamonds are conditional routers. Dotted edges are their branches, taken in the numbered priority order. The completed prefix is fixed and only the unrun tail is rewritten.

A Appendix

A.1 Uncertainty for Table 1

The per-model accuracy CIs in Table 1 (and the per-model rates in Table 2) come from an empirical percentile bootstrap. For each model we dump the per-question 0/1 records over the held-out test split and resample them with replacement 10,000 times, recording the mean of each resample. The 95% CI is the 2.5th and 97.5th percentiles of that distribution. The resampling RNG is seeded (seed 42) so the intervals are exactly reproducible. Since greedy decoding makes each checkpoint’s predictions deterministic, this captures the sampling uncertainty of estimating a rate from a finite test set rather than any generation randomness.

Table 2: Per-model full-split rates for all three metrics, as percentages with 95% percentile bootstrap CIs in brackets (10,000 iterations, seed 42). Rows match Table 1. Step is the number of completed GRPO steps and Seed the training seed. The base model is untrained (—). The accuracy column reproduces the main table. The within-10% and strict-format columns are reported only here.

Model	Step	Seed	Accuracy (exact numeric)	Partial (within 10%)	Format (strict format)
Base <code>gemma-3-1b-it</code>	—	—	47.38 [44.81, 50.11]	49.96 [47.31, 52.69]	4.02 [3.03, 5.08]
Baseline GRPO ($K=2$)	5864	42	0.08 [0.00, 0.23]	0.38 [0.08, 0.76]	2.81 [1.90, 3.71]
$K=8$	5864	42	56.03 [53.45, 58.76]	58.98 [56.41, 61.64]	94.84 [93.63, 96.06]
$K=8$ + reweighted reward	5864	42	58.53 [55.88, 61.18]	60.80 [58.15, 63.46]	94.31 [93.03, 95.53]

A.2 Normalised reward component decomposition

This is the diagnostic referenced in the I.3 discussion of the reward-component reweighting experiment. It provides supplementary evidence about how the logged component scores change after the intervention.

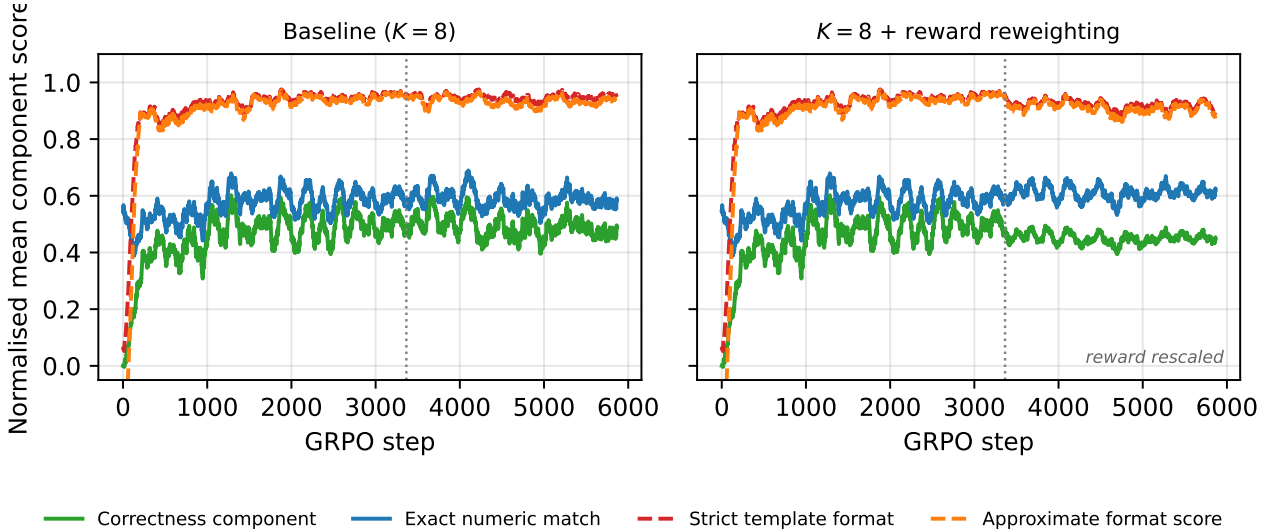


Figure 4: Normalised reward component diagnostic for the two $K=8$ runs. The left panel uses the baseline reward and the right panel uses reward reweighting. Each curve is the logged mean component score divided by its verified maximum. The vertical axis is therefore a normalised mean component score rather than a behavioural frequency. Only `check_numbers/1.5` can be interpreted directly as an exact numeric match rate. The dotted line marks the step 3364 branch. Strict template format is near saturation while the normalised correctness component score remains near one half. Differences in the logged component magnitudes largely reflect the altered reward scale, while the normalised trajectories show little movement after the branch.

A.3 Improving the baseline experiments

This appendix collects the full set of improvement experiments. The aim of this work was exploratory. Instead of committing to a single improvement, we trialled a broad range of modifications to the baseline and measured their effect, to identify which direction was most promising to develop further. The improvement reported in the main text was selected on the basis of these trials.

The experiments begin with the K sweep experiments that vary the number of generations per prompt K directly, followed by the experiments run at $K = 2$ and at $K = 8$. Each part has a results table and one training plot. Every run is identified by its Weights & Biases (W&B) run ID and git commit so it can be recovered. Unless noted, the Accuracy column is exact numeric match on the 64 example greedy GSM8K test suite at the step 3364 checkpoint with seed 42.

Multiple experiment runs were made possible by utilising the TPUs set up by Dr Boris Bolliet and by using personal TPUs provided by Google Cloud Console.

K -sweep experiments

Table 3: K -sweep (NUM_GENERATIONS) experiments. One row per run, all other hyperparameters at baseline values, seed 42.

K	W&B ID	Accuracy	Wall-clock (h:mm)	Commit
2	jgs4c6k1	2/64 (3.12%)	4:42	7e696c4
4	x4j7yhdP	35/64 (54.69%)	6:38	19e78cf
8	m3xp6k97	38/64 (59.38%)	7:25	99059ce
16 (step 2991)	xqbl406c	36/64 (56.25%) ^f	8:02	17c90d5

^f $K = 16$ terminated from a TPU out of memory error at step 2991. The step-2500 checkpoint is the last evaluated.

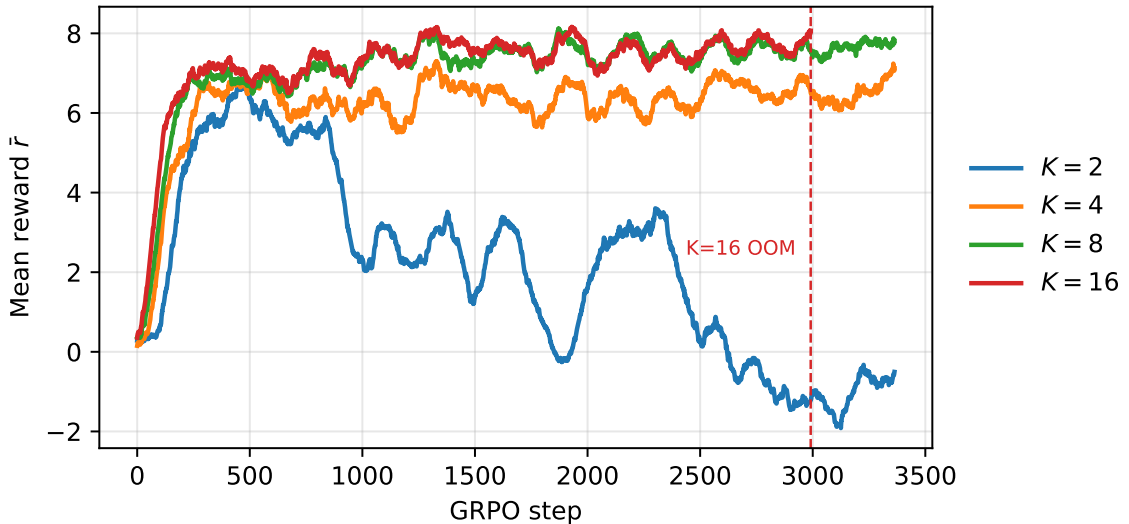


Figure 5: Training mean reward vs. step for $K \in \{2, 4, 8, 16\}$ (centred moving average). The curves show training stability only, $K = 2$ destabilises and decays, while $K = 4$, $K = 8$ and $K = 16$ all remain stable, $K = 4$ holds a stable plateau a little below the closely-tracking $K = 8$ and $K = 16$ pair. High reward at $K \geq 8$ should not be interpreted as high accuracy, since reward (notably the format component) can rise while exact accuracy does not. The $K = 16$ run terminated from a TPU out of memory error at step 2991, so its trace is truncated rather than converged.

$K = 2$ experiments

Table 4: Final LoRA evaluation for $K = 2$ experiments. Each row is a single change above the $K = 2$ baseline.

Change	W&B ID	Accuracy	Wall-clock (h:mm)	Commit
Baseline ($K = 2$)	jgs4c6k1	2/64 (3.12%)	4:42	7e696c4
$\beta = 0.32$	oet2tfjd	0/64 (0%)	1:07	5e7d8f5
Empty penalty	dsi65u1z	9/64 (14.06%)	6:17	2dac639
LoRA $r=128, \alpha=128$	v5cvlwkm	21/64 (32.81%)	5:29	762bb69
$\beta = 10^{-6}$	8rmv0hgg	33/64 (51.56%)	3:26	5e7d8f5

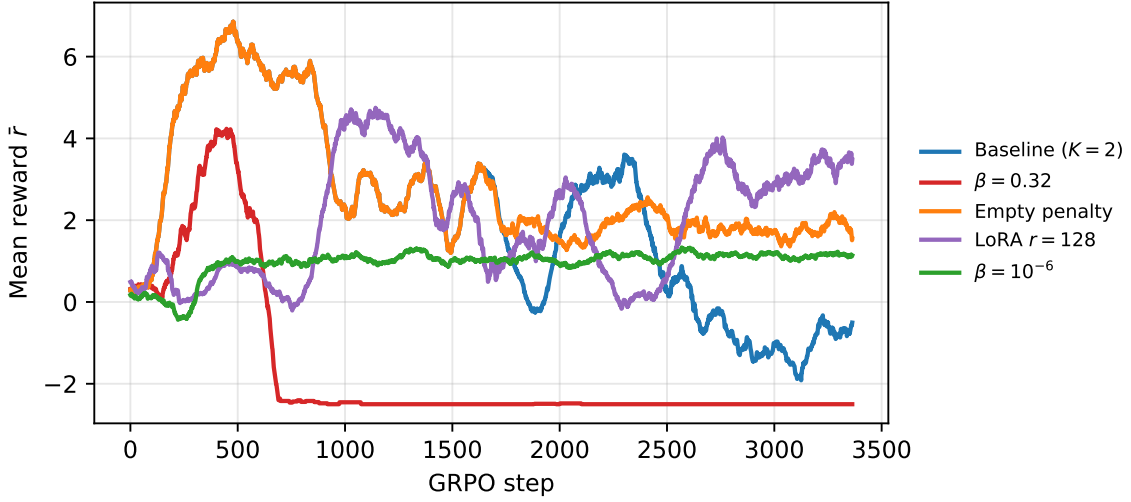


Figure 6: Training mean reward vs. step for the $K = 2$ experiments (centred moving average), one curve per run in the table above. The curves expose the instability of the $K = 2$ regime. The baseline and the $\beta = 0.32$ run destabilise and decay (consistent with their 3.12% and 0% final accuracy), whereas $\beta = 10^{-6}$ holds a steadier reward and avoids the late collapse.

$K = 8$ experiments

Table 5: Final LoRA evaluation for $K = 8$ experiments. Each row is a single change above the $K = 8$ baseline.

Change	W&B ID	Accuracy	Wall-clock (h:mm)	Commit
Baseline ($K = 8$)	m3xp6k97	38/64 (59.38%)	7:25	99059ce
Learning rate 10^{-5}	hozux9t6	19/64 (29.69%)	8:03	99059ce
LoRA $r=128, \alpha=128$	aoz8dtkp	19/64 (29.69%)	8:32	99059ce
Hard+medium subset	6yiowy1y	30/64 (46.88%)	7:35	57c6409
Length penalty	cyay16mj	32/64 (50.00%)	5:17	89cde30
Reweighted reward	smuzmoal	46/64 (71.88%) ^d	7:08	24583db

^d Evaluated at the step-5864 checkpoint.

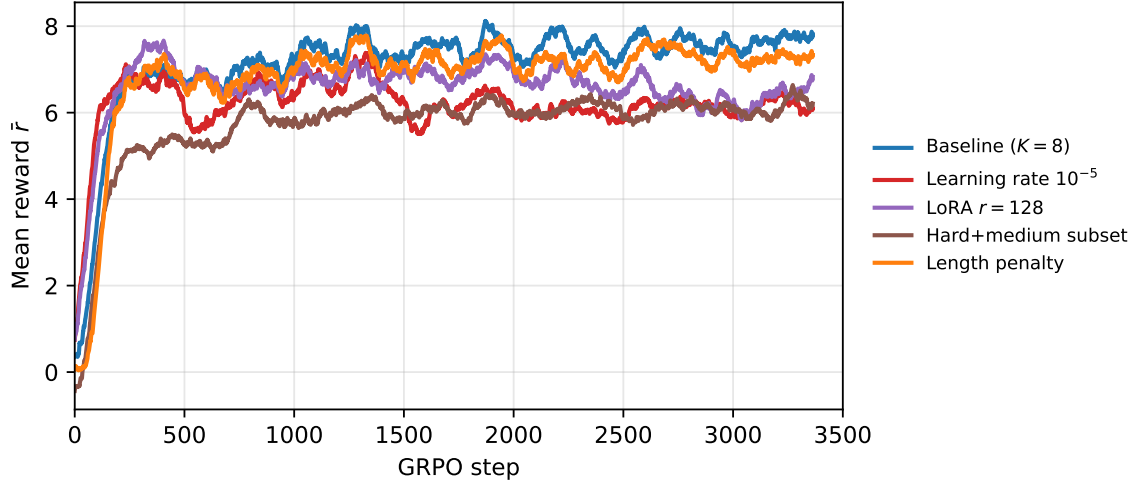


Figure 7: Training mean reward vs. step for the $K = 8$ experiments (centred moving average). Unlike $K = 2$, all runs are stable and remain flat, confirming that the $K = 8$ regime does not destabilise.

The difficult data experiment trains on the harder slices of GSM8K rather than the full distribution, with all other hyperparameters matching the $K = 8$ baseline (seed 42). A base-model mining pass over the GSM8K training split was used to bucket questions by difficulty, yielding 1846 hard, 1990 medium, and 3637 easy questions. The hard and medium buckets form the training subset. The motivation is that concentrating on harder problems might push the model past the easy-question ceiling. In practice it did not improve exact accuracy. Harder problems give a sparser correct-answer reward, so more generation groups are degenerate (all-correct or all-incorrect) and contribute no learning signal, leaving the final accuracy below the $K = 8$ baseline.

A.4 Part I.3 resources

The resources supporting Part I.3 are collated below. Note: There were public access issues to our full Weights and Biases logs. Please request access or refer to our public experiments report below.

- [Team repository \(GitLab\)](#)
- [Summary of all experiments \(W&B report\)](#)
- [TensorBoard logs and checkpoints \(W&B artifacts\)](#)
- [TensorBoard logs \(GitLab\)](#)
- [JSONL for the 1319-question evaluation \(GitLab\)](#)